NAME: AYAS AHAMED

ROLL NO: 2025115162

1. Implement a **Stack using Queue data structure**.
   Code:

```
#include <stdio.h>
#define MAX 100

int queue[MAX];
int front = -1, rear = -1;

/* Enqueue operation */
void enqueue(int value)
{
    if (rear == MAX - 1)
    {
        printf("Queue Overflow\n");
        return;
    }

    if (front == -1)
        front = 0;

    queue[++rear] = value;
}

/* Dequeue operation */
int dequeue()
{
    if (front == -1 || front > rear)
    {
        printf("Queue Underflow\n");
        return -1;
    }

    return queue[front++];
}

/* Push operation - Stack using Queue */
void push(int value)
{
    int i, size;

    size = rear - front + 1;

    enqueue(value);

    /* Move previous elements behind the new element */
    for (i = 0; i < size; i++)
```

```
    {
        enqueue(dequeue());
    }

    printf("%d pushed into stack\n", value);
}

/* Pop operation */
void pop()
{
    int value;

    if (front == -1 || front > rear)
    {
        printf("Stack Underflow\n");
        return;
    }

    value = dequeue();
    printf("%d popped from stack\n", value);

    /* Reset queue when empty */
    if (front > rear)
    {
        front = rear = -1;
    }
}

/* Display stack */
void display()
{
    int i;

    if (front == -1 || front > rear)
    {
        printf("Stack is empty\n");
        return;
    }

    printf("Stack elements are:\n");

    for (i = front; i <= rear; i++)
    {
        printf("%d ", queue[i]);
    }

    printf("\n");
}

int main()
{
```

```
    int choice, value;

    while (1)
    {
        printf("\n--- STACK USING QUEUE ---\n");
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Display\n");
        printf("4. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                printf("Enter value: ");
                scanf("%d", &value);
                push(value);
                break;

            case 2:
                pop();
                break;

            case 3:
                display();
                break;

            case 4:
                return 0;

            default:
                printf("Invalid choice\n");
        }
    }
}
```

Output:

```
--- STACK USING QUEUE ---
1. Push
2. Pop
3. Display
4. Exit

Enter your choice: 1
Enter value: 10
10 pushed into stack

Enter your choice: 1
Enter value: 20
20 pushed into stack
```

```
Enter your choice: 1
Enter value: 30
30 pushed into stack

Enter your choice: 3
Stack elements are:
30 20 10

Enter your choice: 2
30 popped from stack
```

2. Implement a **Queue using Stack data structure**.

Code:

```
#include <stdio.h>
#define MAX 100

int stack1[MAX], stack2[MAX];
int top1 = -1, top2 = -1;

/* Push into Stack 1 */
void push1(int value)
{
    if (top1 == MAX - 1)
    {
        printf("Queue Overflow\n");
        return;
    }

    stack1[++top1] = value;
}

/* Push into Stack 2 */
void push2(int value)
{
    stack2[++top2] = value;
}

/* Pop from Stack 1 */
int pop1()
{
    return stack1[top1--];
}

/* Pop from Stack 2 */
int pop2()
{
    return stack2[top2--];
}

/* Enqueue operation */
```

```
void enqueue(int value)
{
    push1(value);
    printf("%d inserted into queue\n", value);
}

/* Dequeue operation */
void dequeue()
{
    int value;

    if (top1 == -1 && top2 == -1)
    {
        printf("Queue Underflow\n");
        return;
    }

    /* Transfer elements from Stack 1 to Stack 2 */
    if (top2 == -1)
    {
        while (top1 != -1)
        {
            push2(pop1());
        }
    }

    value = pop2();

    printf("%d deleted from queue\n", value);
}

/* Display queue */
void display()
{
    int i;

    if (top1 == -1 && top2 == -1)
    {
        printf("Queue is empty\n");
        return;
    }

    printf("Queue elements are: ");

    /* Elements in Stack 2 */
    for (i = top2; i >= 0; i--)
    {
        printf("%d ", stack2[i]);
    }

    /* Elements in Stack 1 */
```

```
    for (i = 0; i <= top1; i++)
    {
        printf("%d ", stack1[i]);
    }

    printf("\n");
}

int main()
{
    int choice, value;

    while (1)
    {
        printf("\n--- QUEUE USING STACK ---\n");
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Display\n");
        printf("4. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                printf("Enter value: ");
                scanf("%d", &value);
                enqueue(value);
                break;

            case 2:
                dequeue();
                break;

            case 3:
                display();
                break;

            case 4:
                return 0;

            default:
                printf("Invalid choice\n");
        }
    }

    return 0;
}
```

Output:

```
--- QUEUE USING STACK ---
```

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value: 15
15 inserted into queue

--- QUEUE USING STACK ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value: 65
65 inserted into queue

--- QUEUE USING STACK ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value: 85
85 inserted into queue

--- QUEUE USING STACK ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Queue elements are: 15 65 85

--- QUEUE USING STACK ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
15 deleted from queue

3. Write a C program using **Stack data structure** to implement **managing the history of web pages visited**.
Code:

```
#include <stdio.h>
#include <string.h>

#define MAX 100

char history[MAX][100];
```

```
int top = -1;

/* Visit a new web page */
void visitPage(char page[])
{
    if (top == MAX - 1)
    {
        printf("History is full!\n");
        return;
    }

    top++;
    strcpy(history[top], page);

    printf("Visited: %s\n", page);
}

/* Go back to previous page */
void goBack()
{
    if (top <= 0)
    {
        printf("No previous page available.\n");
        return;
    }

    printf("Going back from: %s\n", history[top]);
    top--;

    printf("Current page: %s\n", history[top]);
}

/* Display browsing history */
void displayHistory()
{
    int i;

    if (top == -1)
    {
        printf("History is empty.\n");
        return;
    }

    printf("\nBrowsing History:\n");

    for (i = top; i >= 0; i--)
    {
        printf("%s\n", history[i]);
    }
}
```

```
int main()
{
    int choice;
    char page[100];

    while (1)
    {
        printf("\n--- WEB PAGE HISTORY ---\n");
        printf("1. Visit Page\n");
        printf("2. Go Back\n");
        printf("3. Display History\n");
        printf("4. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                printf("Enter web page: ");
                scanf("%s", page);
                visitPage(page);
                break;

            case 2:
                goBack();
                break;

            case 3:
                displayHistory();
                break;

            case 4:
                printf("Exiting...\n");
                return 0;

            default:
                printf("Invalid choice!\n");
        }
    }

    return 0;
}
```

Output:

```
--- WEB PAGE HISTORY ---
1. Visit Page
2. Go Back
3. Display History
4. Exit

Enter your choice: 1
```

```
Enter web page: google.com
Visited: google.com

Enter your choice: 1
Enter web page: youtube.com
Visited: youtube.com

Enter your choice: 1
Enter web page: github.com
Visited: github.com

Enter your choice: 3

Browsing History:
github.com
youtube.com
google.com

Enter your choice: 2
Going back from: github.com
Current page: youtube.com
```

4. Write a C program to **sort a stack using a temporary stack**.

Code:

```
#include <stdio.h>

#define MAX 100

int stack[MAX], temp[MAX];
int top = -1, tempTop = -1;

/* Push into original stack */
void push(int value)
{
    if (top == MAX - 1)
    {
        printf("Stack Overflow\n");
        return;
    }

    stack[++top] = value;
}

/* Pop from original stack */
int pop()
{
    return stack[top--];
}

/* Push into temporary stack */
void tempPush(int value)
{
```

```
    temp[++tempTop] = value;
}

/* Pop from temporary stack */
int tempPop()
{
    return temp[tempTop--];
}

/* Sort the stack */
void sortStack()
{
    int value;

    while (top != -1)
    {
        value = pop();

        /* Move larger elements back to original stack */
        while (tempTop != -1 && temp[tempTop] > value)
        {
            push(tempPop());
        }

        tempPush(value);
    }

    /* Copy sorted elements back to original stack */
    while (tempTop != -1)
    {
        push(tempPop());
    }
}

/* Display stack */
void display()
{
    int i;

    printf("Stack elements:\n");

    for (i = top; i >= 0; i--)
    {
        printf("%d ", stack[i]);
    }

    printf("\n");
}

int main()
{
```

```
    int n, i, value;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter stack elements:\n");

    for (i = 0; i < n; i++)
    {
        scanf("%d", &value);
        push(value);
    }

    printf("\nOriginal Stack:\n");
    display();

    sortStack();

    printf("\nSorted Stack:\n");
    display();

    return 0;
}
```

Output:

```
Enter number of elements: 5
Enter stack elements:
30 10 50 20 40
Original Stack:
40 20 50 10 30

Sorted Stack:
10 20 30 40 50
```

5. **5. Stock Span Problem**

The **Stock Span Problem** is a financial problem where we have a series of daily price quotes for a stock, denoted by an array.
The task is to calculate the **span of the stock's price for all days**.
The span of a stock's price for the **i-th day** gives the **maximum number of consecutive days leading up to the i-th day (including the current day)** where the stock price was **less than or equal to the price on day i**.
Implement using **Stack**.

Example:

```
Input:  [100, 80, 60, 70, 60, 75, 85]
Output: [1,   1,  1,  2,  1,  4,  6]
```

Code:

```
#include <stdio.h>

#define MAX 100

int stack[MAX];
```

```
int top = -1;

void push(int value)
{
    stack[++top] = value;
}

int pop()
{
    return stack[top--];
}

int main()
{
    int price[MAX], span[MAX];
    int n, i;

    printf("Enter number of days: ");
    scanf("%d", &n);

    printf("Enter stock prices:\n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &price[i]);
    }

    /* First day always has span 1 */
    span[0] = 1;
    push(0);

    for (i = 1; i < n; i++)
    {
        /*
         * Remove days whose price is less than
         * or equal to the current price
         */
        while (top != -1 && price[stack[top]] <= price[i])
        {
            pop();
        }

        /*
         * If stack is empty, all previous days
         * have prices <= current price
         */
        if (top == -1)
        {
            span[i] = i + 1;
        }
        else
        {
```

```
            span[i] = i - stack[top];
        }

        push(i);
    }

    printf("\nStock Prices: ");
    for (i = 0; i < n; i++)
    {
        printf("%d ", price[i]);
    }

    printf("\nStock Span:   ");
    for (i = 0; i < n; i++)
    {
        printf("%d ", span[i]);
    }

    printf("\n");

    return 0;
}
```

Output:

```
Enter number of days: 7
Enter stock prices:
100 80 60 70 60 75 85
Stock Prices: 100 80 60 70 60 75 85
Stock Span:   1 1 1 2 1 4 6
```

6. **Nearest Greater Element to the Right**

Given an array of elements, for each element find the **nearest element to its right that has a higher value than the current element**.
If no such element exists, set the value to **-1**.
Implement using **Stack**.

Example:

```
Input:  [2, 1, 1, 3, 2, 1]
Output: [3, 3, 3, -1, -1, -1]
```

Code:

```
#include <stdio.h>

#define MAX 100

int stack[MAX];
int top = -1;

void push(int value)
{
    stack[++top] = value;
}
```

```
int pop()
{
    return stack[top--];
}

int main()
{
    int arr[MAX], result[MAX];
    int n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter array elements:\n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    /*
     * Traverse from right to left
     */
    for (i = n - 1; i >= 0; i--)
    {
        /* Remove elements smaller than or equal to current */
        while (top != -1 && stack[top] <= arr[i])
        {
            pop();
        }

        /* If stack is empty, no greater element exists */
        if (top == -1)
        {
            result[i] = -1;
        }
        else
        {
            result[i] = stack[top];
        }

        /* Push current element */
        push(arr[i]);
    }

    printf("\nInput:  [");
    for (i = 0; i < n; i++)
    {
        printf("%d", arr[i]);

        if (i < n - 1)
            printf(", ");
```

```
    }
    printf("]\n");

    printf("Output: [");
    for (i = 0; i < n; i++)
    {
        printf("%d", result[i]);

        if (i < n - 1)
            printf(", ");
    }
    printf("]\n");

    return 0;
}
```

Output:

```
Enter number of elements: 6
Enter array elements:
2 1 1 3 2 1
Input:  [2, 1, 1, 3, 2, 1]
Output: [3, 3, 3, -1, -1, -1]
```

7. Minimum Number Following I/D Pattern

Given a pattern containing only **I's and D's**:

- I → Increasing
- D → Decreasing

Devise an algorithm using **Stack** to print the **minimum number following that pattern**.

Digits are from **1–9**, and digits **cannot repeat**.

Example:

```
Input:  D
Output: 21

Input:  DIDI
Output: 21435
```

Code:

```
#include <stdio.h>
#include <string.h>

#define MAX 100

int stack[MAX];
int top = -1;

/* Push an element into stack */
void push(int value)
{
    stack[++top] = value;
}
```

```
/* Pop an element from stack */
int pop()
{
    return stack[top--];
}

int main()
{
    char pattern[MAX];
    int n, i, j;

    printf("Enter I/D pattern: ");
    scanf("%s", pattern);

    n = strlen(pattern);

    printf("Minimum number: ");

    for (i = 0; i <= n; i++)
    {
        /*
         * Push the next digit
         */
        push(i + 1);

        /*
         * If we reach the end or find I,
         * pop all elements from stack.
         */
        if (i == n || pattern[i] == 'I')
        {
            while (top != -1)
            {
                printf("%d", pop());
            }
        }
    }

    printf("\n");

    return 0;
}
```

Output:

```
Enter I/D pattern: D
Minimum number: 21

Enter I/D pattern: DIDI
Minimum number: 21435
```

8. Minimum Sum of Two Numbers Formed from Digits

Given an array of digit values from **0 to 9**, find the **minimum possible sum of two numbers formed from the digits of the array**.
All digits of the given array must be used to form the two numbers.
Use **Queue** for implementation.
Example:
Input: [6, 8, 4, 5, 2, 3]

Numbers formed: 358 and 246

Output: 604

Code:

```
#include <stdio.h>

#define MAX 100

int queue[MAX];
int front = 0, rear = -1;

/* Insert into queue */
void enqueue(int value)
{
    if (rear == MAX - 1)
    {
        printf("Queue Overflow\n");
        return;
    }

    queue[++rear] = value;
}

/* Remove from queue */
int dequeue()
{
    if (front > rear)
        return -1;

    return queue[front++];
}

/* Sort the queue */
void sortQueue()
{
    int i, j, temp;

    for (i = front; i <= rear; i++)
    {
        for (j = i + 1; j <= rear; j++)
        {
            if (queue[i] > queue[j])
            {
```

```
            temp = queue[i];
            queue[i] = queue[j];
            queue[j] = temp;
        }
    }
  }
}

int main()
{
    int n, i, digit;
    int num1 = 0, num2 = 0;
    int turn = 0;

    printf("Enter number of digits: ");
    scanf("%d", &n);

    printf("Enter the digits (0-9):\n");

    for (i = 0; i < n; i++)
    {
        scanf("%d", &digit);
        enqueue(digit);
    }

    /* Sort digits in ascending order */
    sortQueue();

    /*
     * Distribute digits alternately
     * between the two numbers.
     */
    while (front <= rear)
    {
        digit = dequeue();

        if (turn == 0)
        {
            num1 = num1 * 10 + digit;
            turn = 1;
        }
        else
        {
            num2 = num2 * 10 + digit;
            turn = 0;
        }
    }

    printf("\nNumber 1: %d", num1);
    printf("\nNumber 2: %d", num2);
    printf("\nMinimum Sum: %d\n", num1 + num2);
```

```
    return 0;
}
```
Output:
Enter number of digits: 6
Enter the digits (0-9):
6 8 4 5 2 3

Number 1: 248
Number 2: 356
Minimum Sum: 604

9. First Negative Integer in Every Window
Given an array and a positive integer K, find the **first negative integer for each contiguous subarray (window) of size K**.
If a window does not contain a negative integer, print **0** for that window.
Implement using **Queue**.
Example:
Input: [-8, 2, 3, -6, 1], K = 2

Windows:
[-8, 2] → -8
[2, 3] → 0
[3, -6] → -6
[-6, 1] → -6

Output: [-8, 0, -6, -6]

Code:
```
#include <stdio.h>

#define MAX 100

int queue[MAX];
int front = 0, rear = -1;

void enqueue(int index)
{
    queue[++rear] = index;
}

void dequeue()
{
    front++;
}

int main()
{
    int arr[MAX];
    int n, k;
    int i;
```

```
printf("Enter number of elements: ");
scanf("%d", &n);

printf("Enter array elements:\n");
for (i = 0; i < n; i++)
{
   scanf("%d", &arr[i]);
}

printf("Enter window size K: ");
scanf("%d", &k);

if (k > n || k <= 0)
{
   printf("Invalid window size\n");
   return 0;
}

printf("\nOutput: [");

for (i = 0; i < n; i++)
{
   /* Add negative element's index to queue */
   if (arr[i] < 0)
   {
      enqueue(i);
   }

   /* Remove indices outside the current window */
   if (front <= rear && queue[front] <= i - k)
   {
      dequeue();
   }

   /* Start printing when first window is formed */
   if (i >= k - 1)
   {
      if (front <= rear)
         printf("%d", arr[queue[front]]);
      else
         printf("0");

      if (i < n - 1)
         printf(", ");
   }
}

printf("]\n");

return 0;
```

```
}
```

Output:

Enter number of elements: 5
Enter array elements:
-8 2 3 -6 1
Enter window size K: 2
Output: [-8, 0, -6, -6]

10. Rotten Oranges

Given a matrix of dimensions **m × n**, where each cell can have values 0, 1, or 2:

- 0 → Empty cell
- 1 → Cell containing a **fresh orange**
- 2 → Cell containing a **rotten orange**

A rotten orange at (i, j) can rot fresh oranges that are its **neighbours: up, down, left and right**.
Find the **minimum time required for all oranges to become rotten**.
Use **Queue** for implementation.

Example:

Input:
2 1 0 2 1
1 0 1 2 1
1 0 0 2 1

Output: 2

Code:

```
#include <stdio.h>

#define MAX 100

struct Cell
{
    int row;
    int col;
};

struct Cell queue[MAX * MAX];
int front = 0, rear = -1;

void enqueue(int row, int col)
{
    queue[++rear].row = row;
    queue[rear].col = col;
}

struct Cell dequeue()
{
    return queue[front++];
}
```

```
int main()
{
    int m, n;
    int matrix[MAX][MAX];
    int i, j;
    int fresh = 0;
    int time = 0;

    /* Directions: up, down, left, right */
    int dr[] = {-1, 1, 0, 0};
    int dc[] = {0, 0, -1, 1};

    printf("Enter rows and columns: ");
    scanf("%d %d", &m, &n);

    printf("Enter the matrix:\n");

    for (i = 0; i < m; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &matrix[i][j]);

            if (matrix[i][j] == 2)
            {
                /* Add all rotten oranges to queue */
                enqueue(i, j);
            }
            else if (matrix[i][j] == 1)
            {
                fresh++;
            }
        }
    }

    /* BFS */
    while (front <= rear && fresh > 0)
    {
        int size = rear - front + 1;

        /* Process all rotten oranges of current minute */
        for (i = 0; i < size; i++)
        {
            struct Cell current = dequeue();

            for (j = 0; j < 4; j++)
            {
                int newRow = current.row + dr[j];
                int newCol = current.col + dc[j];

                /* Check valid neighbour */
```

```
            if (newRow >= 0 && newRow < m &&
                newCol >= 0 && newCol < n &&
                matrix[newRow][newCol] == 1)
            {
                /* Fresh orange becomes rotten */
                matrix[newRow][newCol] = 2;

                fresh--;

                enqueue(newRow, newCol);
            }
        }
    }

    time++;
  }

  if (fresh > 0)
    printf("\nOutput: -1\n");
  else
    printf("\nOutput: %d\n", time);

  return 0;
}
```

Output:

```
Enter rows and columns: 3 5

Enter the matrix:
2 1 0 2 1
1 0 1 2 1
1 0 0 2 1
Output: 2
```